

Data Mining Practical – Association Rules Mining

Association rule mining is a technique to identify underlying relations between different items. Take an example of a Super Market where customers can buy variety of items. Usually, there is a pattern in what the customers buy. For instance, mothers with babies buy baby products such as milk and diapers. In short, transactions involve a pattern. More profit can be generated if the relationship between the items purchased in different transactions can be identified.

For instance, if item A and B are bought together more frequently then several steps can be taken to increase the profit. For example:

1. A and B can be placed together so that when a customer buys one of the products they don't have to go far away to buy the other product.
2. Both A and B can be packaged together, etc.

The process of identifying an association between these products is called association rule mining. The goal of today's practical is to:

- Use a simple market dataset to explain the basic concepts of association rules mining
- We will then apply these concepts to the BlackFriday dataset, explore the association rules, explain the findings, and visualize the results

1. Mining Associations – Basic Practice

Code: First, let's take a look at the dataset

```
import pandas as pd

data = pd.read_csv('Sample Data.csv',header='infer')

data_copy = data.copy()

data
```

Output:

	transID	Apple	Bread	Coke	MP3	Egg
0	100	1	?	1	1	?
1	200	?	1	1	?	1
2	300	1	1	1	?	1
3	400	?	1	?	?	1

Each row of the dataset represents items that were purchased together on the same day at the same store. The occurrence of an item in a transaction is represented by a Boolean value – 1. The absence of an item is depicted by a missing value, the character '?'.

To make use of the apriori module we will be using - the mlxtend library - we need to convert the dataset according to it's liking. The apriori module requires a quantitative dataframe that has either 0 and 1 or True and False as data.

Code: We are very close to this already, so we simply need to perform some preprocessing and remove the 'transID' attribute, and change the '?' characters to 0.

```
data = data.drop('transID', axis=1)
data = data.replace('?', 0)
data
```

Output:

	Apple	Bread	Coke	MP3	Egg
0	1	0	1	1	0
1	0	1	1	0	1
2	1	1	1	0	1
3	0	1	0	0	1

Code: We then need to convert our attributes to the numerical datatype

```
data['Apple'] = pd.to_numeric(data['Apple'])
data['Bread'] = pd.to_numeric(data['Bread'])
data['Coke'] = pd.to_numeric(data['Coke'])
data['MP3'] = pd.to_numeric(data['MP3'])
data['Egg'] = pd.to_numeric(data['Egg'])
```

Code: The current mlxtend needs a Boolean (True/False) datatype

```
# apriori need |True,False
data['Apple'] = data['Apple'].astype(bool)
data['Bread'] = data['Bread'].astype(bool)
data['Coke'] = data['Coke'].astype(bool)
data['MP3'] = data['MP3'].astype(bool)
data['Egg'] = data['Egg'].astype(bool)
```

1.2 Apriori Algorithm for Association Rule Mining

There are three major components of Apriori algorithm:

- Support
- Confidence
- Lift

We will explain these three concepts with the help of an example.

Suppose we have a record of 1 thousand customer transactions, and we want to find the Support, Confidence, and Lift for two items e.g., burgers and ketchup. Out of one thousand transactions, 100 contain ketchup while 150 contain a burger. Out of 150 transactions where a burger is purchased, 50 transactions contain ketchup as well. Using this data, we want to find the support, confidence, and lift.

1. Support

Support refers to the default popularity of an item and can be calculated by finding number of transactions containing a particular item divided by total number of transactions. Suppose we want to find support for item B. This can be calculated as:

$$\text{Support}(B) = (\text{Transactions containing } (B))/(\text{Total Transactions})$$

For instance, if out of 1000 transactions, 100 transactions contain Ketchup then the support for item Ketchup can be calculated as:

$$\text{Support}(\text{Ketchup}) = (\text{Transactions containing Ketchup})/(\text{Total Transactions})$$

$$\begin{aligned}\text{Support}(\text{Ketchup}) &= 100/1000 \\ &= 10\%\end{aligned}$$

2. Confidence

Confidence refers to the likelihood that an item B is also bought if item A is bought. It can be calculated by finding the number of transactions where A and B are bought together, divided by total number of transactions where A is bought. Mathematically, it can be represented as:

$$\text{Confidence}(A \rightarrow B) = (\text{Transactions containing both } (A \text{ and } B))/(\text{Transactions containing } A)$$

Coming back to our problem, we had 50 transactions where Burger and Ketchup were bought together. While in 150 transactions, burgers are bought. Then we can find likelihood of buying ketchup when a burger is bought can be represented as confidence of Burger $\rightarrow$ Ketchup and can be mathematically written as:

$$\text{Confidence}(\text{Burger} \rightarrow \text{Ketchup}) = (\text{Transactions containing both } (\text{Burger and Ketchup}))/(\text{Transactions containing A})$$

$$\begin{aligned}\text{Confidence}(\text{Burger} \rightarrow \text{Ketchup}) &= 50/150 \\ &= 33.3\%\end{aligned}$$

3. Lift

Lift(A → B) refers to the increase in the ratio of sale of B when A is sold. Lift(A → B) can be calculated by dividing Confidence(A → B) divided by Support(B). Mathematically it can be represented as:

$$\text{Lift}(A \rightarrow B) = (\text{Confidence}(A \rightarrow B)) / (\text{Support}(B))$$

Coming back to our Burger and Ketchup problem, the Lift(Burger → Ketchup) can be calculated as:

$$\text{Lift}(\text{Burger} \rightarrow \text{Ketchup}) = (\text{Confidence}(\text{Burger} \rightarrow \text{Ketchup})) / (\text{Support}(\text{Ketchup}))$$

$$\begin{aligned}\text{Lift}(\text{Burger} \rightarrow \text{Ketchup}) &= 33.3/10 \\ &= 3.33\end{aligned}$$

Lift basically tells us that the likelihood of buying a Burger and Ketchup together is 3.33 times more than the likelihood of just buying the ketchup. A Lift of 1 means there is no association between products A and B. Lift of greater than 1 means products A and B are more likely to be bought together. Finally, Lift of less than 1 refers to the case where two products are unlikely to be bought together.

1.2 Applying Apriori

There are 4 major steps that we must do when using the Apriori algorithm:

1. We must set a minimum value for support and confidence. This means that we are only interested in finding rules for the items that have certain default existence (e.g., support) and have a minimum value for co-occurrence with other items (e.g., confidence).
2. We need to extract all the frequent subsets having higher value of support than minimum threshold, or where the rule indicates that many people do both X and Y
3. Select all the rules from the subsets with confidence value higher than minimum threshold, or where the rule indicates that if someone does X, there is a good chance they will also do Y
4. Order the rules by descending order of Lift.

Code: To implement this, we must first ensure we have the mlxtend library installed, as currently the scikit-learn library does not support ARM. Please refer to the first practical for installing packages – the distribution name is just ‘mlxtend’.

```
# importing relevant libraries
import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, association_rules
import matplotlib.pyplot as plt

freq_items = apriori(data, min_support=0.5, use_colnames=True, verbose=1)
freq_items.head(10)
```

When creating our frequent itemsets, a number of parameters should be set within apriori(). The min_support parameter refers to the minimum support threshold. For this example, let us set it at 0.5, or 50%, because we want to find some relatively strong associations, but feel free to play around with this value. Another parameter is the max_len, which has been purposefully left out here, but it controls the max length of the itemset generated. If not set, simply all possible rules are evaluated.

Use `_colnames` and `verbose` are used to make the end dataframe more readable by preserving the column names and showing us how many iterations were performed.

Output:

	support	itemsets
0	0.50	(Apple)
1	0.75	(Bread)
2	0.75	(Coke)
3	0.75	(Egg)
4	0.50	(Coke, Apple)
5	0.50	(Bread, Coke)
6	0.75	(Bread, Egg)
7	0.50	(Egg, Coke)
8	0.50	(Bread, Egg, Coke)

Here we can see from the output, that there is a total of 8 frequent itemsets from our small dataset. The downward closure property states that any subset of a frequent itemset must also be frequent. For example, if the itemset {Coke, Apple} is frequent, then each subset must also be frequent, so {Coke} and {Apple} both appear as frequent itemsets as well.

Now that we have our frequent itemsets, we want to find out what our association rules are. For this, we need to select a minimum confidence value, which for this case we will set to 90%. In other words, when product X is purchased, we can say that the purchase of product Y is 90% or more.

Code: The minimum confidence is set as a fraction against the parameter `min_threshold` after selecting 'confidence' for the metric parameter. The metric property can be set to confidence, lift, support, leverage or conviction.

```
rules = association_rules(freq_items, metric="confidence", min_threshold=0.9)
rules.head()
```

Output:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction
0	(Apple)	(Coke)	0.50	0.75	0.50	1.0	1.333333	0.1250	inf
1	(Egg)	(Bread)	0.75	0.75	0.75	1.0	1.333333	0.1875	inf
2	(Bread)	(Egg)	0.75	0.75	0.75	1.0	1.333333	0.1875	inf
3	(Egg, Coke)	(Bread)	0.50	0.75	0.50	1.0	1.333333	0.1250	inf
4	(Coke, Bread)	(Egg)	0.50	0.75	0.50	1.0	1.333333	0.1250	inf

For example, if we examine the first row of our dataframe;

- The probability of seeing {Apple} sales is seen as 50% (the antecedent support)
- The {Coke} intake is seen as 75% (the consequent support)
- We can say that the support of both is measured as 50%

- With a confidence value of 1.0, we can say that out of all the transactions containing {Apple} 100% of the transactions contain {Coke} as well.
- Finally, a lift of 1.33 shows that {Coke} is 1.33 times more likely to be bought by customers that buy {Apple}, compared to its default sale.

As a result, as items {Apple} and {Coke} are bought together more frequently, then several steps can be taken to increase the profit, such as changing the shop layout so that the items are together, creating bundle deals, etc.

2. Mining Associations – BlackFriday Dataset

Now that we have an understanding of how to find strong rules using a simple dataset, let's try and apply it to the BlackFriday dataset. First let's take a look at the dataset:

```
data = pd.read_csv('BlackFriday.csv',header='infer')

data_copy = data.copy()

data
```

Output:

	User_ID	Lat	Long	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
0	1000001	0.0433	0.6300	P00069042	F	0-17	10	A	2	0	3	NaN	NaN	8370
1	1000001	0.0467	0.7033	P00248942	F	0-17	10	A	2	0	1	6.0	14.0	15200
2	1000001	0.0500	0.6833	P00087842	F	0-17	10	A	2	0	12	NaN	NaN	1422
3	1000001	0.0500	0.6900	P00085442	F	0-17	10	A	2	0	12	14.0	NaN	1057
4	1000002	0.0500	0.6933	P00285442	M	55+	16	C	4+	0	8	NaN	NaN	7969
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1995	1000319	0.5867	0.4200	P00194142	F	51-55	6	B	3	1	11	16.0	NaN	4662
1996	1000319	0.5867	0.4433	P00120042	F	51-55	6	B	3	1	1	2.0	NaN	15179
1997	1000319	0.5900	0.3600	P00304642	F	51-55	6	B	3	1	2	14.0	15.0	6659
1998	1000319	0.5900	0.3633	P00326742	F	51-55	6	B	3	1	11	NaN	NaN	5947
1999	1000319	0.5900	0.4300	P00067942	F	51-55	6	B	3	1	5	14.0	NaN	5245

Once again, we will need to apply some preprocessing steps. First we must consider which variables we want to use for our association rules mining. There are many different combinations we could use, but we must consider that in order to use ARM, we must transform the variable into categorical form, which could be problematic for variables such as Product_ID which have many unique values. For such a variable, if we were to binarize it – it could create thousands of additional columns which is undesirable.

For this example, let us assume that we are only interested in finding out the associations between different product categories and how it relates to the purchase values. As the 'Purchase' attribute is currently numeric, we must discretize it into bins. We must also handle the NaN values in the product category attributes. We could simply drop the rows containing missing values, but this would lead to a lot of information loss. As the product category attributes represent the different categories a particular product falls into, NaN represents a valid value i.e., where a product only belongs to one Product_Category, we should fill this value in with a user-defined constant. For this we will simply use the number 0.

```
data = data[['Product_Category_1', 'Product_Category_2', 'Product_Category_3',
'Purchase']] # Create subset including only desired attributes
data['Purchase'] = pd.qcut(data['Purchase'], 3, labels=[0, 1, 2])
data = data.fillna(0)
data
```

Output:

	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
0	3	0.0	0.0	1
1	1	6.0	14.0	2
2	12	0.0	0.0	0
3	12	14.0	0.0	0
4	8	0.0	0.0	1
...	...	...	...	...
1995	11	16.0	0.0	0
1996	1	2.0	0.0	2
1997	2	14.0	15.0	0
1998	11	0.0	0.0	0
1999	5	14.0	0.0	0

To make our results more readable, we can replace the labels of the 'Purchase' attribute to be more meaningful i.e. 0 = Low, 1 = Medium, 2 = High.

```
data['Purchase'] = data['Purchase'].replace(0, 'Low')
data['Purchase'] = data['Purchase'].replace(1, 'Medium')
data['Purchase'] = data['Purchase'].replace(2, 'High')
data
```

Output:

	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
0	3	0.0	0.0	Medium
1	1	6.0	14.0	High
2	12	0.0	0.0	Low
3	12	14.0	0.0	Low
4	8	0.0	0.0	Medium
...	...	...	...	...
1995	11	16.0	0.0	Low
1996	1	2.0	0.0	High
1997	2	14.0	15.0	Low
1998	11	0.0	0.0	Low
1999	5	14.0	0.0	Low

We can then check to see what our current unique categories are. These will form our new column names once we perform the next step.

```
items = set()
for col in data:
    items.update(data[col].unique())
print(items)
```

Output:

```
{0.0, 1, 2, 3, 4, 5, 6, 7, 8, 9.0, 10, 11, 12, 13, 14, 15, 16, 17, 18, 'Medium',
'High', 'Low'}
```

For Apriori algorithm, this dataset needs to be one-hot encoded. This is similar to using `pd.get_dummies()`, where a new column will be generated for each unique category in our dataset, where each column consists of TRUE/FALSE or 1/0 values that indicate whether that item was included in the transaction.

```
df = data.copy()
for item_id in items:
    print(f'item_id={item_id}')
    df[item_id] = (
        (df['Product_Category_1'] == item_id) |
        (df['Product_Category_2'] == item_id) |
        (df['Product_Category_3'] == item_id) |
        (df['Purchase'] == item_id)
    ).astype(int)
df
```

Output:

	Product_Category_1	Product_Category_2	Product_Category_3	Purchase	0	1	10	11	12	13	...	3	4	5	6	7	8	9
0	3	0	0	Medium	1	0	0	0	0	0	...	1	0	0	0	0	0	0
1	1	6	14	High	0	1	0	0	0	0	...	0	0	0	1	0	0	0
2	12	0	0	Low	1	0	0	0	1	0	...	0	0	0	0	0	0	0
3	12	14	0	Low	1	0	0	0	1	0	...	0	0	0	0	0	0	0
4	8	0	0	Medium	1	0	0	0	0	0	...	0	0	0	0	0	1	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1995	11	16	0	Low	1	0	0	1	0	0	...	0	0	0	0	0	0	0
1996	1	2	0	High	1	1	0	0	0	0	...	0	0	0	0	0	0	0
1997	2	14	15	Low	0	0	0	0	0	0	...	0	0	0	0	0	0	0
1998	11	0	0	Low	1	0	0	1	0	0	...	0	0	0	0	0	0	0
1999	5	14	0	Low	1	0	0	0	0	0	...	0	0	1	0	0	0	0

Now, let's select only the new one-hot (binarized) encoded columns.

```
ohe_df = df[items] # Selecting all items as columns
# ohe_df <--- one-hot-encoded
ohe_df
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning:
  will not call `transform_cell` automatically in the future. Please pass the result
 ument and any exception that happen during the transform in `preprocessing_exc_tup
  ove.
  and should_run_async(code)
```

	0	1	10	11	12	13	14	15	16	17	...	3	4	5	6	7	8	9	High	Low	Medium
0	1	0	0	0	0	0	0	0	0	0	...	1	0	0	0	0	0	0	0	0	1
1	0	1	0	0	0	0	1	0	0	0	...	0	0	0	1	0	0	0	1	0	0
2	1	0	0	0	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	1	0

Note that you may see read warning messages (throughout this prac), ignore them.

Now we can apply the apriori module from the mlxtend library to get a list of our frequent items. Here we are using a minimum support of 20%.

```
freq_items = apriori(ohe_df, min_support = 0.2, use_colnames = True, verbose = 1)
freq_items
```

Output:

	support	itemsets
0	0.2765	(1)
1	0.3455	(5)
2	0.3670	(8)
3	0.3335	(Low)
4	0.3335	(High)
5	0.6905	(0.0)
6	0.3330	(Medium)
7	0.2275	(1, High)
8	0.2555	(0.0, 5)
9	0.2810	(0.0, 8)
10	0.2735	(0.0, Low)
11	0.2705	(0.0, Medium)

Using these frequent items, we can then generate our association rules with a minimum confidence level of 50%.

```
rules = association_rules(freq_items, metric="confidence", min_threshold=0.5)
rules
```

Output:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction
0	(1)	(High)	0.2765	0.3335	0.2275	0.822785	2.467121	0.135287	3.760964
1	(High)	(1)	0.3335	0.2765	0.2275	0.682159	2.467121	0.135287	2.276295
2	(5)	(0.0)	0.3455	0.6905	0.2555	0.739508	1.070975	0.016932	1.188136
3	(8)	(0.0)	0.3670	0.6905	0.2810	0.765668	1.108860	0.027587	1.320773
4	(Low)	(0.0)	0.3335	0.6905	0.2735	0.820090	1.187676	0.043218	1.720304
5	(Medium)	(0.0)	0.3330	0.6905	0.2705	0.812312	1.176412	0.040564	1.649016

What conclusions can we make from this? Are there any useful rules?

- 6 rules are found to be strong, but they are not entirely useful. "1 -> High" with a confidence of 82% indicates that out of all transactions containing (1), 82% of them also contain a purchase value of (High). Now this may just indicate that the purchase category of (1) represents a high value item category such as technology, jewelry, etc. The others containing (0) are also not entirely useful because it represents that when a product is purchased i.e. with a product category of 5, then usually it does not belong to any other product categories.

Try to change some parameters, such as setting the minimum support to 10%, with a confidence of 50%. What kind of results does this give? Are any of them meaningful?

3. Mining Associations – FP-Growth

FP-Growth is an unsupervised machine learning technique used for association rule mining which is faster than apriori. However, it cannot be used on large datasets due to its high memory requirements. Implementation is very similar to Apriori, and it produces the same results, but for very large datasets it may not be able to efficiently handle it, and computation times may be high.

```
from mlxtend.frequent_patterns import fpgrowth

freq_items = fpgrowth(ohe_df, min_support=0.2, use_colnames=True)
freq_items
```

Output:

	support	itemsets
0	0.6905	(0.0)
1	0.3330	(Medium)
2	0.3335	(High)
3	0.2765	(1)
4	0.3335	(Low)
5	0.3670	(8)
6	0.3455	(5)
7	0.2705	(0.0, Medium)
8	0.2275	(1, High)
9	0.2735	(0.0, Low)
10	0.2810	(0.0, 8)
11	0.2555	(0.0, 5)

```
rules = association_rules(freq_items, metric="confidence", min_threshold=0.5)
rules
```

Output:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction
0	(Medium)	(0.0)	0.3330	0.6905	0.2705	0.812312	1.176412	0.040564	1.649016
1	(1)	(High)	0.2765	0.3335	0.2275	0.822785	2.467121	0.135287	3.760964
2	(High)	(1)	0.3335	0.2765	0.2275	0.682159	2.467121	0.135287	2.276295
3	(Low)	(0.0)	0.3335	0.6905	0.2735	0.820090	1.187676	0.043218	1.720304
4	(8)	(0.0)	0.3670	0.6905	0.2810	0.765668	1.108860	0.027587	1.320773
5	(5)	(0.0)	0.3455	0.6905	0.2555	0.739508	1.070975	0.016932	1.188136